Rent■Finder Platform
Full System Design Document

Prepared by Picours Technology Company
Production■Grade System Design
Version 1.0 • NAD Currency • Namibia

1. System Purpose & Scope

The Rent■Finder platform enables busy individuals to delegate rental searches to verified finders through a secure escrow system. The system ensures that funds are protected, payouts are deliberate, and platform revenue is guaranteed.

2. Design Principles

• Escrow■first money flow
• Deterministic financial outcomes
• No automatic fund movement
• Admin■controlled payouts
• Auditable and transparent operations

3. High■Level Architecture

The platform consists of frontend clients, a stateless Node.js API server, and a Supabase■managed PostgreSQL database. All critical financial invariants are enforced at the database level.

4. User Roles & Access

Busy Individuals manage tasks and escrows. Rent Finders complete tasks and receive payouts. Admins control disputes, payouts, and finance with strict role enforcement and logging.

5. Escrow System Design

Escrow is created after task acceptance. Payment moves escrow to held state. Resolution paths include release, cancellation, or dispute. Escrow status determines allowable actions.

6. Financial Model

Client pays 250 NAD. Finder receives 200 NAD upon escrow release. Platform retains 50 NAD service fee in all outcomes. This rule is enforced at schema, logic, and reporting layers.

7. Payout Subsystem

Payouts represent entitlement execution, not escrow resolution. Only one payout is allowed per escrow. Admins manually progress payout states to accommodate bank and mobile money constraints.

8. Audit & Transparency

All financial actions emit immutable audit logs capturing who acted, what changed, and when. Users can view escrow timelines derived from these logs.

9. Security Architecture

Security layers include JWT authentication, role checks, rate■limited endpoints, idempotent financial logic, and strict database constraints.

10. Failure Handling

The system safely handles payment failures, admin mistakes, retries, and partial outages without losing or duplicating funds.

11. Scalability Considerations

Stateless API design and database■level invariants allow horizontal scaling. Future automation requires configuration, not redesign.

12. Deployment Architecture

The platform supports separate development and production environments with secure secret management and automated backups.

Conclusion

This system design prioritizes financial safety, transparency, and operational control, making it suitable for real■world deployment in Namibia and beyond.